Experiment 4: Implement K-Nearest Neighbors (KNN) and evaluate model performance

Aim: Implement K-Nearest Neighbors (KNN) and evaluate model performance.

Theory:

1. Dataset Source

Dataset Name: **Dermatology Dataset – Classification**

Source: Kaggle

Dataset Link:
https://www.kaggle.com/datasets/olcaybolat1/dermatology-dataset-classification

This dataset is suitable for multi-class classification and contains numerical attributes, making it appropriate for KNN.

2. Dataset Description

The Dermatology dataset is a multi-class classification dataset used to diagnose different skin diseases.

Dataset Characteristics:

- Total Instances: 366
- Total Features: 34 input attributes
- Target Variable: class
- Number of Classes: 6
- Attribute Type: Integer numerical values
- Some missing values present

Features:

The features represent clinical and histopathological characteristics such as:

- Erythema
- Scaling
- Itching
- Follicular papules

- Fibrosis
- Age

Each feature ranges between 0 and 3 indicating severity levels.

Target Variable:

The class attribute represents six types of skin diseases, making this a **multi-class classification problem**.

3. Mathematical Formulation of KNN

K-Nearest Neighbors (KNN) is a supervised, instance-based learning algorithm.

It does not build an explicit model. Instead, it stores the training data and makes predictions based on distance calculations.

Step 1: Distance Calculation

The most commonly used distance metric is **Euclidean Distance**:

$$d(x, x_i) = \sqrt{\sum_{j=1}^{n} (x_j - x_{ij})^2}$$

Where:

- x = test sample
- x_i = training sample
- n = number of features

Step 2: Select K Nearest Neighbors

The algorithm selects the **K smallest distances**.

Step 3: Majority Voting (Classification)

The predicted class is determined by:

$$\hat{y} = \text{Majority Class among K neighbors}$$

Important Concept: Choosing K

- Small K → High variance (overfitting)
- Large K → High bias (underfitting)

Optimal K is usually found using experimentation or cross-validation.

4. Algorithm Limitations

Although KNN is simple and effective, it has several limitations:

- Computationally expensive for large datasets
- Sensitive to irrelevant features
- Performance degrades in high-dimensional data (Curse of Dimensionality)
- Requires feature scaling
- Slow during prediction time

When KNN May Not Perform Well:

- Very large datasets
- High-dimensional sparse data
- Imbalanced datasets
- Real-time prediction systems requiring fast inference

5. Methodology

Step 1: Data Collection

Download dataset from Kaggle.

Step 2: Data Preprocessing

- Replace missing values
- Convert to numeric format
- Handle NaN values
- Separate features and target

Step 3: Feature Scaling

Apply StandardScaler because KNN is distance-based.

Step 4: Train-Test Split

Split dataset into:

- 70% Training
- 30% Testing

Step 5: Model Training

Apply KNeighborsClassifier with selected K value.

Step 6: Prediction

Predict class labels for test data.

Step 7: Model Evaluation

Evaluate using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-Score

6. Performance Analysis

Accuracy:

$$Accuracy = \frac{CorrectPredictions}{TotalPredictions}$$

Confusion Matrix:

Shows class-wise prediction performance.

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

F1-Score:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Expected Results:

- Accuracy typically between 95% – 99% depending on K value.
- Proper feature scaling significantly improves accuracy.
- Performance varies with different K values.

7. Hyperparameter Tuning

The main hyperparameters of KNN are:

- n_neighbors (K value)
- metric (distance metric)
- weights (uniform or distance-based)

Example Tuning:

- Try K values from 1 to 20
- Select K with highest validation accuracy

Impact of Tuning:

- Improves generalization
- Reduces overfitting
- Enhances stability of predictions

Cross-validation or GridSearchCV can be used for systematic tuning.

Code:

```
# Import Required Libraries

import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load the Dataset

# Read the dermatology dataset
data = pd.read_csv("/content/dermatology_database_1.csv")

# Replace '?' values with NaN (missing values)
data.replace('?', np.nan, inplace=True)

# Convert all columns to numeric data type
data = data.apply(pd.to_numeric)

# Fill missing values with column mean
data.fillna(data.mean(), inplace=True)

# Separate Features and Target Variable

# X contains all input features
X = data.drop("class", axis=1)

# y contains target class labels
y = data["class"]

# Train-Test Split

# Split dataset into 70% training and 30% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Feature Scaling (Important for KNN)
```

```
# Initialize StandardScaler
scaler = StandardScaler()

# Fit and transform training data
X_train = scaler.fit_transform(X_train)

# Transform testing data
X_test = scaler.transform(X_test)

# Apply K-Nearest Neighbors (KNN)

# Create KNN classifier with K = 5
knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train, y_train)

# Make predictions on test data
y_pred = knn.predict(X_test)

# Model Evaluation

# Print Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Print Confusion Matrix
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Print Classification Report (Precision, Recall, F1-score)
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Hyperparameter Tuning (Finding Best K)

accuracy_list = []  # Store accuracy for different K values

# Try K values from 1 to 20
for k in range(1, 21):
```

```
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    accuracy_list.append(accuracy_score(y_test, pred))
# Plot Accuracy vs K Value

plt.plot(range(1, 21), accuracy_list)
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("Accuracy vs K Value")
plt.show()

# Display Best K Value

print("Best Accuracy:", max(accuracy_list))
print("Best K:", accuracy_list.index(max(accuracy_list)) + 1)
```

Output:

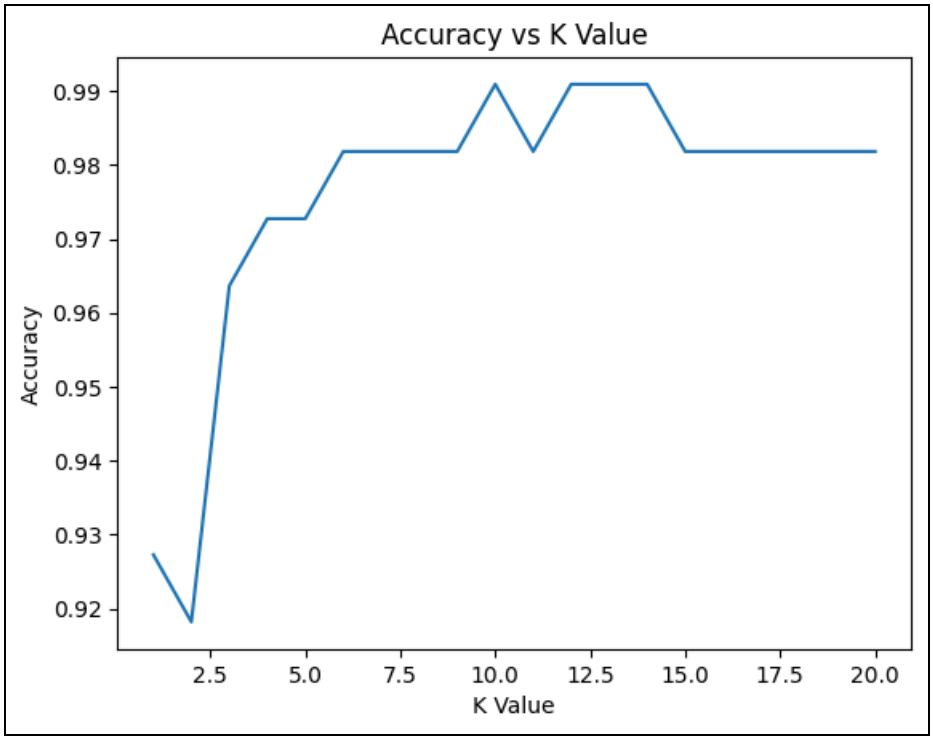
```
Accuracy: 0.9727272727272728

Confusion Matrix:
 [[40  0  0  0  0  0]
 [ 0 15  0  1  0  0]
 [ 0  0 19  0  0  0]
 [ 0  2  0 13  0  0]
 [ 0  0  0  0 16  0]
 [ 0  0  0  0  0  4]]

Classification Report:
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        40
           2       0.88      0.94      0.91        16
           3       1.00      1.00      1.00        19
           4       0.93      0.87      0.90        15
           5       1.00      1.00      1.00        16
           6       1.00      1.00      1.00         4

    accuracy                           0.97       110
   macro avg       0.97      0.97      0.97       110
weighted avg       0.97      0.97      0.97       110
```



Conclusion:

The K-Nearest Neighbors (KNN) algorithm performed very effectively on the Dermatology dataset, achieving a high accuracy of **0.991** with an optimal K value of 10. Proper feature scaling and hyperparameter tuning significantly improved the model's performance. Although KNN provides strong classification results for structured numerical data, its computational cost increases with larger datasets. Overall, KNN proved to be a simple yet powerful classification method for this experiment.